

# Projet Robot

Durieux Lisa-Marie Dekerle Lucas

# Détails du Contrat

01

# Contrat 03

L'objectif principal de notre projet est de programmer un robot que nous pouvons contrôler à distance via Bluetooth. Ce robot doit pouvoir se déplacer et changer de direction (tourner) sur commande.

# Cahier des charges

## Commandes via appli mobile :

- **avancer :**
  - 1er appui → avance
  - 2e appui → arrêt
- **tourner :**
  - si en mouvement → tourne +45°
  - appui long → tourne sur place
  - après chaque rotation → reprend l'avance

## Contraintes fonctionnelles :

- **start :** active/éteint le robot
- Ne peut tourner que si "**avancer**" a été activé
- Avance toujours après une rotation

## Spécifications mouvement :

- Vitesse : 20 cm/s
- Angle rotation : +45° par appui
- Précision :  $\pm 5\%$
- Asservissement en vitesse

# Périphériques choisis

02

Initialisation GPIOs  
 timer6 en base de temps 5ms  
 ADC en single 8 bits sur Vbatt  
 init Tim2 en PWM à 2 kHz  
 uart2\_init 115200 baud  
 receive uart in Buffer\_RX

SET Rc= 2 =ARR/CCR  
 DIR1 SET  
 DIR2 SET

dir = 0, recoit\_commande = 0  
 avance=0  
 tourner=0  
 start = 0  
 Vbatt = 0, Tbatt = 0

tourner = 0, time\_tourne = 0  
 tourne\_court = 0, time\_court = 0

$$\text{Periode} = \text{ARR} * \text{PSC} * \text{Timer}_{clk}$$

$$\text{PSC}_{Opti} \geq \left( \frac{\text{Periode}}{2^{16} * \text{Timer}_{clk}} \right)$$

$$\text{ARR} = \left( \frac{\text{Periode}}{\text{PSC} * \text{Timer}_{clk}} \right) \quad \text{Calculs}$$

## Timer 2 pour PWM

- > Channel 1 & Channel 4
- > f = 2kHz
- > PSCopti >= 0,61 -> PSC=1
- > ARR = 40 000
- > Valeurs réglées avec  
\_\_HAL\_TIM\_SET\_COMPARE,

## Timer 6 pour les interruptions périodiques

- > Période choisie de 5ms
- > PSCopti >= 6,1 -> PSC =7
- > ARR = 57143
- > HAL\_TIM\_PeriodElapsedCallback  
 appelée à la fin de chaque période

## Timer 3 et 4 pour les encodeurs

- > ARR au maximum et PSC=0
- > Encoder mode TI1 and TI2

## ADC1 pour la mesure de tension

- > Résolution 12 bits
- > Quantum q= 0.805mV
- > Connecté au pin PC5

## UART3 pour la commande Bluetooth

- > Réception des commandes  
 toutes les 5ms
- > BaudRate = 9600
- > Connecté aux pins PC11 (TX) &  
 PB10 (RX)

## UART2 pour la communication série

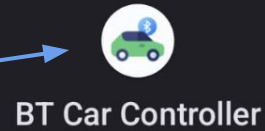
- > Pour le **Debug**
- > BaudRate = 115200
- > TX au PA2 & RX au PA3

# Interfaces utilisées

03

## Interface homme-machine

Application mobile android



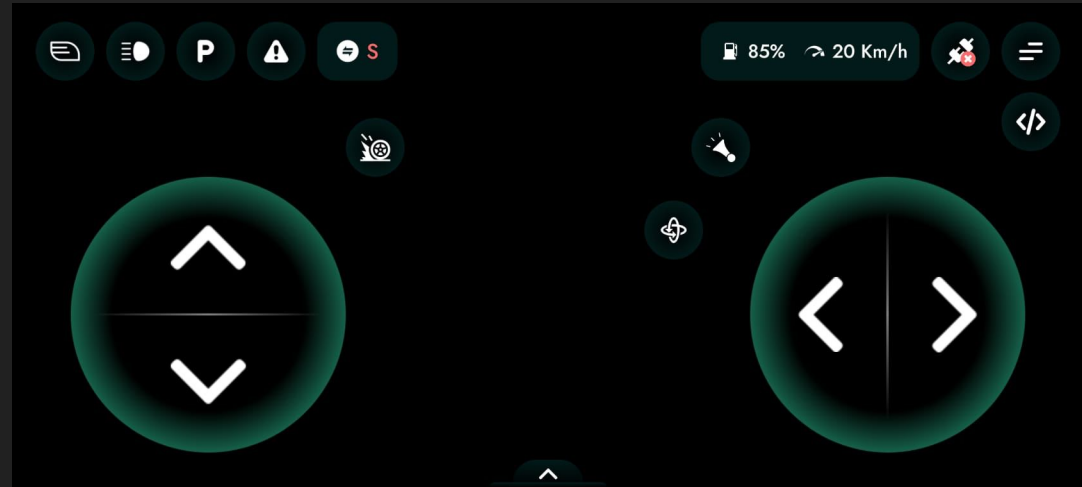
-> BaudRate de 9600

-> Communication via l'UART3  
avec le module Bluetooth

-> Transmission des  
commandes en continu

## Interface de communication interne

UART, Timer, ADC (cf. périphériques  
utilisés)



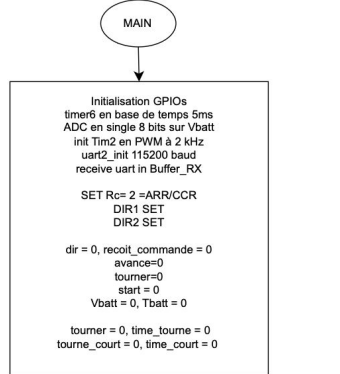


# Conception

Organigramme

04

Vérification  
batterie



PWM OFF  
dir=0

START ?

start = 1

Tbatt > 50

ADC\_Start\_IT

ADC on

Vbatt = get\_result

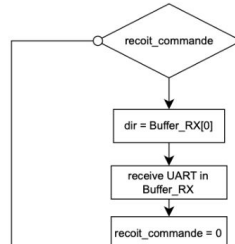
Vbatt < seuil\_batt

LED OFF

LED ON

Tbatt = 0

Réception  
UART



Réception  
de "avance"

dir == AVANCE

!avance

PWM OFF

avance

PWM ON

dir = 0

Réception  
de "tourner"

dir == DROITE

avance && !tourne

tourner = 1  
time\_tourne = 0

dir = 0

Appui court  
sur "droite"

1 < time\_tourne < APPUI\_LONG  
&& tourner  
&& Buffer\_RX[0] == STOP

!tourne\_court  
time\_court = 0

tourne\_court && time\_court <  
TEMPS\_VIRAGE

DIR1 SET

DIR1 RESET

tourne\_court = 0  
tourner = 0  
dir = 0

Appui long  
sur "droite"

time\_tourne > APPUI\_LONG  
&& tourner

DIR1 SET

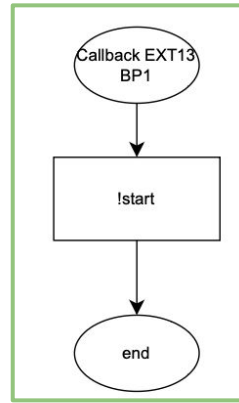
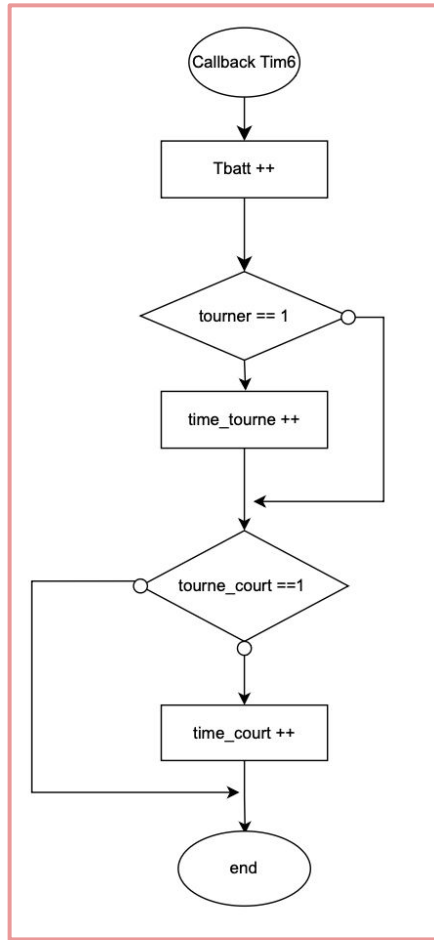
Buffer\_RX[0] == DROITE

tourner = 0

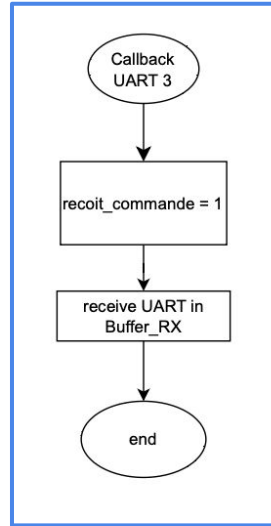
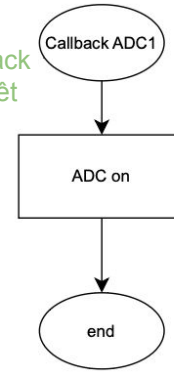
DIR1 RESET

dir = 0

## Timers



Callback d'arrêt



Réception UART

# Fonctionnement & code source

05

# Allumage & Arrêt d'urgence

Le bouton B1 est configurée en interruption EXTI sur le pin PC13..

Lorsqu'il est appuyé l'état de la variable start bascule.

Si start = 1, le robot peut avancer, recevoir des commandes, etc

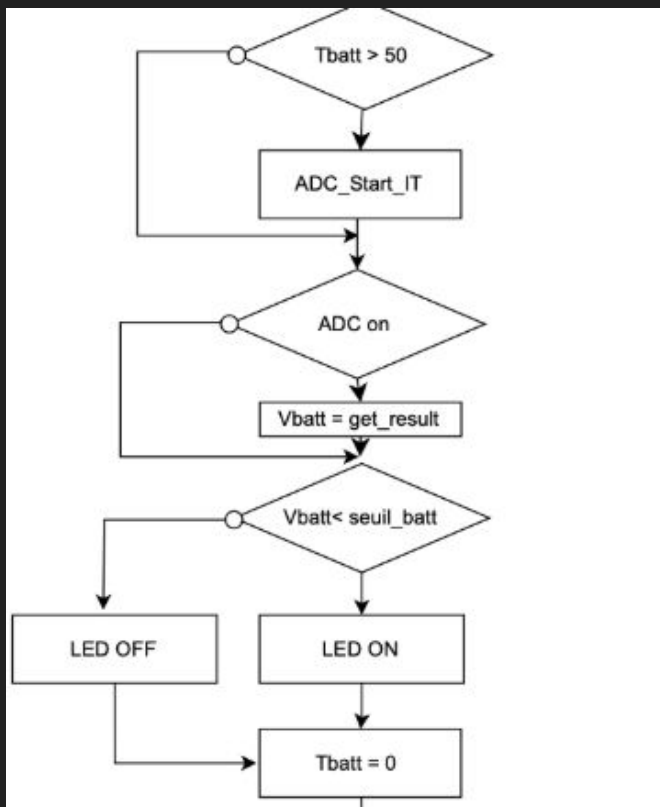
Si start = 0, le robot est arrêté, qu'il soit en mouvement en rotation ou à l'arrêt. Toutes les variables sont remises à 0.

Un filtre anti-rebond a été implémenté pour éviter que le robot ne reçoive un seul appui comme plusieurs.

```
841 void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin){
842
843     if (GPIO_Pin == B1_Pin){
844         if (time_start > 70){
845             start =!start;
846             time_start = 0;
847         }
848     }
849 }
```

```
253 /* USER CODE END 2 */
254
255 /* Infinite loop */
256 /* USER CODE BEGIN WHILE */
257 while (1)
258 {
259     if(start){...
342     else{
343         __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, 0);
344         __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_4, 0);
345         HAL_GPIO_WritePin(DIR1_GPIO_Port, DIR1_Pin, GPIO_PIN_SET);
346         HAL_GPIO_WritePin(DIR2_GPIO_Port, DIR2_Pin, GPIO_PIN_SET);
347         avance = 0;
348         tourner=0;
349         recoit_commande=0;
350         dir=0;
351         HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, RESET); //Eteind LD2
352     }
353     /* USER CODE END WHILE */
354
355     /* USER CODE BEGIN 3 */
356 }
```

# Batterie



```
HAL_ADC_Start_IT(&hadc1);  
if (Tbatt >= 1000){  
    if (Vbatt < seuil_batt) { // Robot déchargé  
        HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_SET);  
    }  
    else { // Robot chargé  
        HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_RESET);  
    }  
    Tbatt = 0; // Remise à zéro du compteur  
}
```

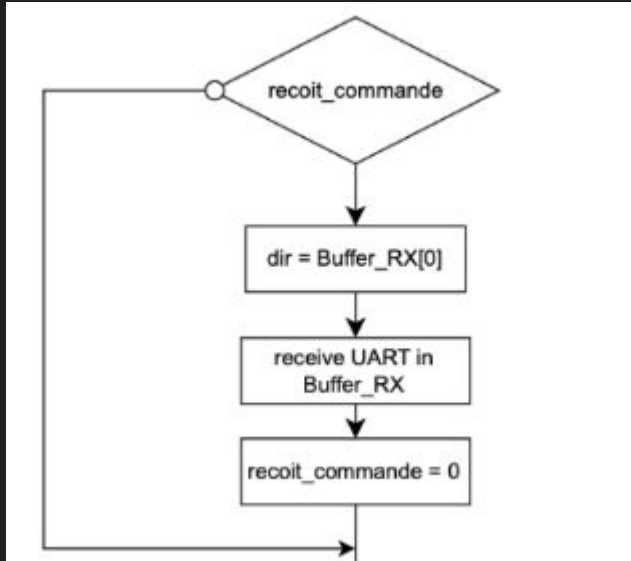
```
818 void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc)  
819 {  
820     if (hadc->Instance == ADC1)  
821     {  
822         uint32_t adc_val = HAL_ADC_GetValue(hadc);  
823         Vbatt = (adc_val/4095.0f)*3.3f;  
824     }  
825 }  
826  
827
```

#define seuil\_batt 3.0f

Avec seuil\_batt = 3.0f

Tbatt est incrémenté par TIM6 (toutes les 5ms)

# Réception de l'UART



```
830 void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
831 {
832
833     if (huart->Instance == USART3) {
834         recoit_commande=1;
835         HAL_UART_Receive_IT(&huart3, (uint8_t *)Buffer_RX, sizeof(Buffer_RX));
836     }
837 }
while (1)
{
    if(start){
        HAL_ADC_Start_IT(&hadc1);
        if (Tbatt >= 1000){ ...

        if(recoit_commande==1){
            dir = Buffer_RX[0]; // stocker la commande reçue
            HAL_UART_Receive_IT(&huart3, (uint8_t *)Buffer_RX, sizeof(Buffer_RX)); // Réarmer la réception
            recoit_commande=0;
        }
    }
}
```

Buffer\_RX[0] est égal aux valeurs 'F', 'R' et 'L' pour respectivement les appuis sur avancer, droite et gauche.

Au relâchement d'une commande, le Buffer\_RX[0] est égal à 'S'.

```
#define AVANCE 'F'
#define STOP 'S'
#define DROITE 'R'
#define GAUCHE 'L'
```

# Commandes de déplacement : avancer

Le robot devait avancer à 20 cm/s  $\pm 5\%$ .

Pour  $R = 50\%$  ( $\rightarrow CCR = 20\,000$ ), la vitesse est trop faible (env 16,5 cm/s)

On calcule donc  $CCR_{exp} = CCR_{th} * V_{th}/V_{exp} = 24\,242$  mais vitesse trop rapide...

Aussi, le robot ne roule pas droit si les deux moteurs sont au même  
\_COMPARE\_

En ajustant les valeurs de chaque moteur, on obtient une vitesse finale de 19,36 cm/s, ce qui respecte le CDC.

```
278  if(dir == AVANCE){
279      avance = !avance; //on change l'état de avancer
280  }
281  if(avance == 1){//le robot était à l'arrêt et la on veut le faire avancer
282      HAL_GPIO_WritePin(DIR2_GPIO_Port, DIR2_Pin, GPIO_PIN_SET);
283      HAL_GPIO_WritePin(DIR1_GPIO_Port, DIR1_Pin, GPIO_PIN_SET);
284
285      __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, 22470);
286      __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_4, 20300);
287  }
288
289  else if(avance == 0){//le robot avançait et la on l'arrête
290      __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, 0);
291      __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_4, 0);
292  }
293  dir = 0;
294  }
```

La variable **avance** bascule à chaque appui sur la touche avancer, ce qui permet de commander le déplacement du robot

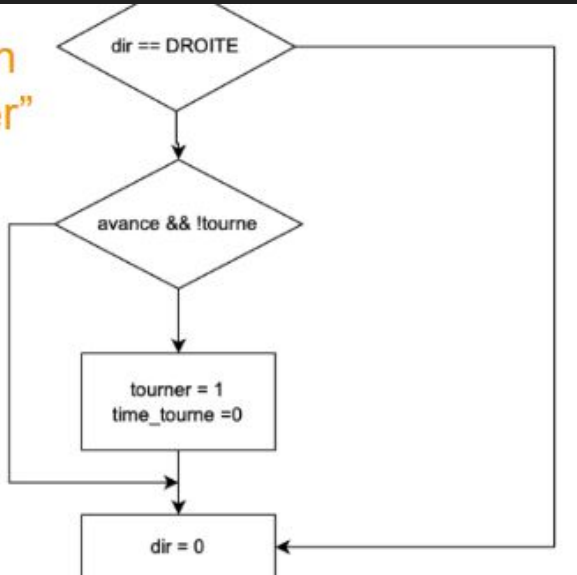
```
#define __HAL_TIM_SET_COMPARE(_HANDLE_, _CHANNEL_, _COMPARE_)
```

Fonction modulant la vitesse d'un moteur



# Commandes de déplacement : tourner

Réception  
de "tourner"



Préliminaire pour les deux manières de tourner

On différencie les deux manières avec time\_tourne ( la durée d'appui sur tourner)

```
269     if (dir == DROITE){
270         if(avance && !tourner){
271             tourner = 1;
272             time_tourne = 0;
273             time_court=0;
274
275         }
276         dir = 0;
277     }
```

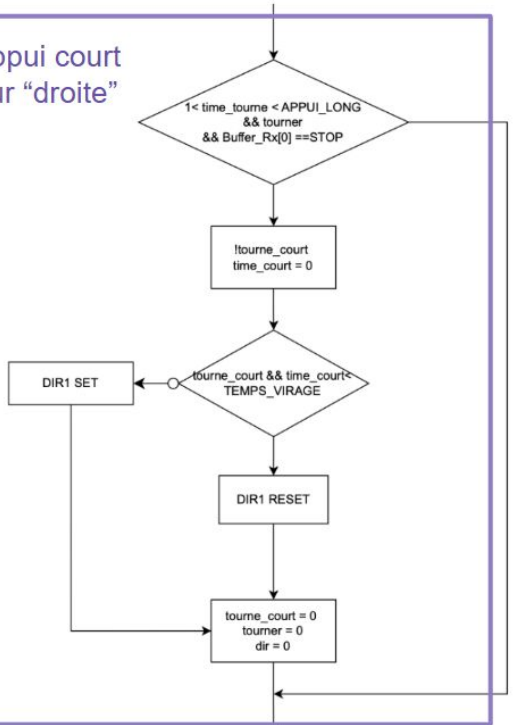
Variable qui permet d'indiquer au programme si le robot doit tourner ou non

Initialisation du compteur de temps de l'appui sur la touche "tourner"

# Le robot tourne à 45°

On tourne tant que `time_court < TEMPS_VIRAGE`

Appui court  
sur "droite"



```
if((time_tourne >1)&& (time_tourne <APPUI_LONG) && tourner && (Buffer_RX[0] == STOP)){ // appui court sur droite

    tourne_court=!tourne_court;
    if(time_court<=TEMPS_VIRAGE ){
        HAL_GPIO_WritePin(DIR1_GPIO_Port, DIR1_Pin, GPIO_PIN_RESET); // moteur gauche recule
        HAL_GPIO_WritePin(DIR2_GPIO_Port, DIR2_Pin, GPIO_PIN_SET); // moteur droit avance
    }
    else{
        tourne_court=0;
        tourner = 0 ;
        HAL_GPIO_WritePin(DIR1_GPIO_Port, DIR1_Pin, GPIO_PIN_SET);
        time_court = 0;
    }

    dir = 0;
}

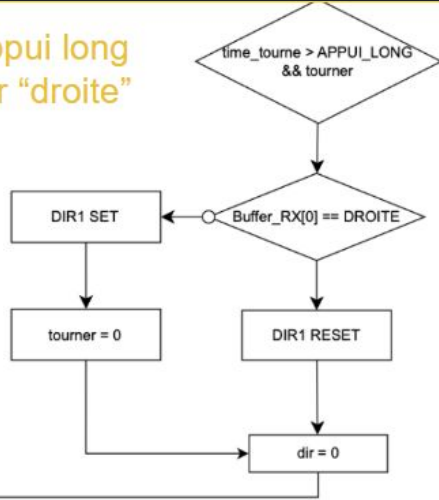
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if(htim->Instance==TIM6){
        T batt++;
        time_start ++;
        if(tourner) time_tourne++; //on incrémente le compteur tant qu'on appuie sur tourner
        if(tourne_court)time_court++; //on incrémente le compteur tant qu'on n'a pas atteint 45°
        if(avance)count_ech++;
    }
}
```

#define TEMPS\_VIRAGE 1/(5\*0.001)

Manière non bloquante d'incrémenter le compteur  
(cad sans while ou HAL\_Delay)

# Le robot tourne sur lui même

Appui long  
sur "droite"



```
331 | if((time_tourne >=APPUI_LONG) && tourner){  
332 |     if(Buffer_RX[0] == DROITE){ //appui long sur droite  
333 |         HAL_GPIO_WritePin(DIR1_GPIO_Port, DIR1_Pin, GPIO_PIN_RESET);  
334 |     }  
335 |     if(Buffer_RX[0] !=DROITE){  
336 |         tourner = 0;  
337 |         HAL_GPIO_WritePin(DIR1_GPIO_Port, DIR1_Pin, GPIO_PIN_SET);  
338 |     }  
339 | }
```

Le robot tourne tant que la touche "droite"  
est maintenue appuyée

On utilise Buffer\_RX[0] au lieu de dir car  
dir =0

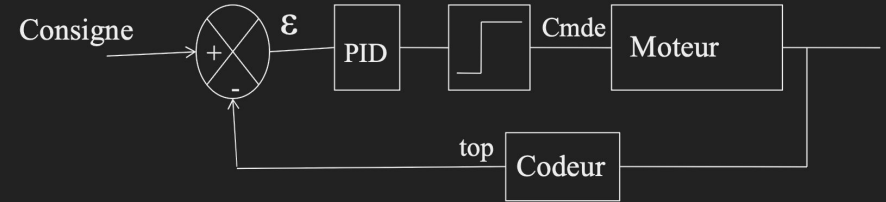
# Asservissement

06

# Objectifs de l'asservissement

Maintenir une vitesse constante malgré les perturbations

Asservissement PID (Proportionnel - Intégrateur - Dérivé)



Loi discrète :

$$u(t) = \underbrace{K_p e(t)}_{\text{Proportional term}} + \underbrace{K_i \int_0^t e(\tau) d\tau}_{\text{Integral term}} + \underbrace{K_d \frac{de(t)}{dt}}_{\text{Derivative term}}$$

```
void calcul_commande(float* cmd_d, float* cmd_g) {

    int32_t tops_d = __HAL_TIM_GET_COUNTER(&htim3);
    int32_t tops_g = __HAL_TIM_GET_COUNTER(&htim4);

    erreur_d = consigne_tops - tops_d + tops_precedents_d;
    erreur_g = consigne_tops - tops_g + tops_precedents_g;

    tops_precedents_d = tops_d;
    tops_precedents_g = tops_g;

    erreur_precedent_d = erreur_d;
    erreur_precedent_g = erreur_g;

    somme_erreur_d += erreur_d;
    somme_erreur_g += erreur_g;

    *cmd_d = Kp * erreur_d + Ki * somme_erreur_d + Kd * (erreur_d - erreur_precedent_d);
    *cmd_g = Kp * erreur_g + Ki * somme_erreur_g + Kd * (erreur_g - erreur_precedent_g);
}
```

# Implémentation de l'asservissement

On calcule la commande de vitesse pour le moteur droit et gauche séparément. Le calcul se fait avec des tops donc on doit convertir la consigne de 20 cm/s en tops/Téch selon la formule suivante:

Tops par période

Tours par secondes

$$\begin{aligned} \text{consigne\_tops} &= \left( \frac{\text{vitesse}_{\text{cm/s}}}{\text{circonference\_roue}_{\text{cm}}} \right) \times \text{tops\_par\_tour\_roue} \times T_{\text{éch}} \\ &= \left( \frac{20}{18.8} \right) \times 333.33 \times 0.1 = 35.46 \end{aligned}$$

```
if(avance){
    if(count_ech >= 20) { //période d'échantillonnage de 100ms
        float cmd_d, cmd_g;
        calcul_commande(&cmd_d, &cmd_g);
        // Conversion vers PWM
        uint32_t pwm_d = cmd_d / 100.0f * 40000.0f;
        uint32_t pwm_g = cmd_g / 100.0f * 40000.0f;
        // PWM minimal pour éviter que le moteur ne bloque
        if (pwm_d < 800) pwm_d = 800;
        if (pwm_g < 800) pwm_g = 800;
        // PWM maximal pour ne pas aller trop vite
        if (pwm_d > 22000) pwm_d = 22000;
        if (pwm_g > 22000) pwm_g = 22000;

        __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_4, pwm_d);
        __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, pwm_g);
        count_ech=0;
    }
}
```

Car cmd\_d et cmd\_g sont dans une plage proche de 0–100

# Performances obtenues & Problèmes rencontrés

07

# Performances obtenues

## Vitesse : précision de +/- 1 cm/s

La vitesse finale du robot, avec les réglages effectués, est de 19,6 cm/s (mesuré sur un parcours de 4m).

## Dépendances

La performance du robot dépendait fortement de son niveau de charge

Tous les robots ne fonctionnait pas pareil : les calibrages devaient être refaits si on changeait de robot -> il fallait garder le même robot pour s'assurer de la continuité du projet (B6)

## Rotation : précision à +/- 2.25°

La mesure de l'angle de rotation était plus difficile, mais un marquage au sol de 45° nous a permis de voir que nos réglages respectent le CDC.

## Asservissement

Permet d'assurer que la vitesse reste au plus proche de 20 cm/s malgré les contraintes de batterie

Le cahier des charges imposait une précision de +/- 5%.



# Problèmes rencontrés

## Fonction tourner

Difficultés pour implémenter la fonction tourner

Angle et durée de rotation au réglage très pointilleux à régler

## Réglage de la vitesse

Difficulté d'avoir 20 cm/s au départ → réglage de la vitesse plutôt expérimental

L'asservissement permet d'avoir la vitesse la plus adaptée

## Dépendances

Les dépendances du robot étaient contournables mais ont posé problème au début

## Vérification de la batterie

Difficulté à régler la fonction au début -> mauvais Channel

## Transmission Bluetooth

Problèmes de transmission des commandes Bluetooth ; la callback n'effectue pas la réception des commandes mais active seulement un flag qui permet la réception des commandes sur l'UART

## Organigramme

Difficulté à mettre en place un algorithme sans claire visibilité du fonctionnement du programme. Les tests sur le robot ont fait énormément changer l'algorithme.

# Conclusion

08

# Bilans

Vitesse demandée : 20cm/s +/- 5% → on obtient une vitesse de 19,6 cm/s ✓

Angle de rotation demandé : 45° par appui → angle de rotation respecté ✓

PWM de 2kHz

Asservissement PID réalisé ✓

L'interface Bluetooth, associée aux périphériques de la carte STM32 du robot, permet un pilotage facile du robot.

L'anti rebond ajouté sur le bouton start pour rendre plus fiable les appuis sur le bouton.

# Améliorations possibles

Ajouter des commandes pour tourner à gauche et reculer

Perfectionner la fonction de rotation pour avoir un angle plus précis

Ajout d'un capteur pour détecter les obstacles